

PROMPT IA 06

Shipping + Tracking + Billing + Notification + Returns

Vertical slice post-commande - Go, PostgreSQL, Kafka, RabbitMQ, carrier adapters, Novu, Postal, Jasmin/SMPP, facturation et remboursement

Objectif : fournir a une IA de developpement un cahier d'execution suffisamment strict pour produire du code compilable, teste, maintenable et coherent avec l'architecture e-commerce.

Priorite : backend authoritative - idempotence - resilience - observabilite - GitOps

1. Role de l'IA

Tu es l'ingenieur principal charge d'implementer le vertical slice post-commande. Tu dois modifier le monorepo existant, reutiliser les conventions du Golden Service Go et ne pas dupliquer les abstractions deja disponibles.

Ne genere pas un prototype jetable. Le resultat doit etre executable, teste, observable et deployable. Corrige les erreurs rencontrees avant de poursuivre.

2. Perimetre fonctionnel

Service	Responsabilite
Shipping	Choix transporteur, options, creation/annulation expedition, colis, etiquette, adapters transporteurs.
Tracking	Timeline logistique, statuts, ETA, ingestion des webhooks transporteurs et reconciliation.
Billing	Factures legales, numerotation, avoirs, snapshot fiscal, generation PDF et historique.
Notification	Decision metier d'envoi, preferences, templates/references et publication de jobs.
Returns	Demande de retour, validation, etiquette retour, reception, restock eventuel et remboursement.

3. Architecture obligatoire

```
order.confirmed.v1
|
+--> Shipping --> Carrier Adapter --> DHL / DPD / UPS / Chronopost / ...
| |
| +--> shipping.created.v1
| |
| +--> Tracking
|
+--> Billing --> RabbitMQ --> PDF Worker --> Object Storage
| |
| +--> invoice.issued.v1
|
+--> Notification --> RabbitMQ --> Novu
+--> Postal
+--> Jasmin / SMPP
+--> Push provider
```

```
Returns:
Customer -> Returns -> Shipping return label -> Tracking -> Receive
-> Inventory restock decision -> Payment refund -> Billing credit note
```

Kafka reste le bus des faits metier durables. RabbitMQ sert aux jobs operationnels : PDF, email, SMS, push, traitement d'etiquettes ou autres taches distribuables. Ne melange pas leurs responsabilites.

4. Contraintes d'architecture

- Backend authoritative : aucune confirmation frontend ne fait foi.
- Une base PostgreSQL logique par microservice ; aucun service n'accede aux tables d'un autre domaine.
- REST externe et gRPC interne doivent appeler les memes use cases.
- Outbox transactionnelle pour les evenements metier.
- Idempotence obligatoire pour consumers, webhooks, creation d'expedition, facture, retour et remboursement.

- OpenBao pour secrets/credentials ; aucun secret longue duree dans Git.
- OpenTelemetry pour traces, metrics et correlation IDs.
- Operations critiques versionnees et auditees.

5. Shipping Service

5.1 Ports metier

```
type Carrier interface {
  Quote(ctx context.Context, req QuoteRequest) ([]Rate, error)
  CreateShipment(ctx context.Context, req CreateShipmentRequest) (Shipment, error)
  CancelShipment(ctx context.Context, shipmentID string) error
  GetLabel(ctx context.Context, shipmentID string) (Label, error)
  Track(ctx context.Context, trackingNumber string) (TrackingSnapshot, error)
}
```

Les adapters transporteurs implementent cette interface. Aucun code DHL/UPS/DPD/etc. ne doit contaminer le domaine Shipping.

5.2 Resilience

Ajouter timeouts, retry budget borne, circuit breaker, rate limiting par provider, health state et fallback vers un autre transporteur uniquement lorsqu'une regle metier l'autorise.

5.3 Evenements

```
shipping.requested.v1
shipping.created.v1
shipping.creation_failed.v1
shipping.cancelled.v1
shipping.label_generated.v1
```

6. Tracking Service

Tracking est proprietaire de la timeline, des statuts, de l'ETA et des evenements de livraison. Il ne cree pas d'expedition.

```
CREATED -> IN_TRANSIT -> OUT_FOR_DELIVERY -> DELIVERED
\-> EXCEPTION
\-> RETURN_TO_SENDER
```

Implementer une machine d'etat explicite. Refuser les transitions impossibles ou les traiter selon une policy documentee.

6.1 Webhooks transporteurs

Endpoints dedies via Kong. Valider signature/authentification quand disponible, timestamp/anti-replay, idempotence, schema, taille, rate limit et audit. Transformer ensuite le message externe en evenement interne.

6.2 Reconciliation

Implementer un worker periodique comparant l'etat interne avec l'API authoritative du transporteur. Une correction automatique n'est autorisee que si elle est deterministe et auditable.

7. Billing Service

Billing est separe de Payment. Il gere la facture legale et les avoirs, pas les transactions bancaires.

```
invoice
invoice_number
order_id
issued_at
currency
```

```
lines[]
tax_snapshot
totals
status
document_object_key
version
```

La facture doit conserver un snapshot immuable des données nécessaires. Une modification future de Product, Pricing ou Tax ne modifie jamais une facture émise.

7.1 PDF asynchrone

```
Billing transaction
|
+--> outbox / invoice.issued.v1
|
+--> RabbitMQ pdf.generate
|
v
PDF Worker
|
v
Object Storage
```

Le job PDF doit être idempotent. Retries avec backoff et DLQ. Ne jamais bloquer la confirmation de commande sur la génération du PDF.

8. Notification Service + Novu

Notification conserve les décisions métier : quel événement déclenche quelle notification, à quel destinataire et avec quelles préférences. Novu reste une couche technique d'orchestration.

```
Business Event
|
Notification Service
|
RabbitMQ
|
Novu
+--> Postal email
+--> Jasmin/SMPP SMS
+--> Push mobile/web
```

Une panne Postal, Jasmin ou Novu ne doit jamais faire échouer Order, Payment, Shipping ou Returns. Les notifications sont hors chemin transactionnel critique.

Ajouter retries, backoff, DLQ, deduplication, correlation_id et statut d'envoi.

9. Returns Service

```
REQUESTED
-> APPROVED
-> LABEL_CREATED
-> IN_TRANSIT
-> RECEIVED
-> INSPECTED
-> REFUND_PENDING
-> REFUNDED
-> CLOSED
```

Branches possibles:

```
REQUESTED -> REJECTED
INSPECTED -> NO_RESTOCK
INSPECTED -> RESTOCK_APPROVED
```

Returns orchestre le workflow mais ne reprend pas l'ownership des autres domaines.

9.1 Coordination

```
Returns
+--> Shipping : etiquette retour
+--> Tracking : suivi retour
+--> Inventory : decision / commande de restock
+--> Payment : remboursement
+--> Billing : avoir
+--> Notification : information client
```

Toute demande de remboursement doit utiliser une idempotency key et verifier l'etat authoritative du Payment Service.

10. Saga de retour / remboursement

```
Return received
|
v
Inspection
|
+-- restock eligible --> Inventory
|
v
Payment.refund
|
+-- FAIL --> retry/reconciliation/manual review
|
v
Billing.credit_note
|
v
Return REFUNDED
|
v
Notification
```

Definir clairement les compensations. Ne jamais inventer une compensation impossible : un remboursement bancaire deja execute n'est pas annule par une simple mise a jour locale.

11. Persistance et SQL

Utiliser PostgreSQL avec pgx + sqlc. SQL versionne, migrations expand-migrate-contract, contraintes DB comme derniere ligne de defense.

Ajouter les indexes necessaires et tester les requetes critiques. Pas de SELECT * dans les chemins sensibles, pas de requete non bornee, pagination par curseur lorsque necessaire.

12. Idempotence

L'idempotence doit couvrir au minimum :

- consommation order.confirmed.v1
- CreateShipment
- webhooks carrier
- generation facture et PDF
- jobs Notification
- demande de retour
- restock

- Payment refund
- creation d'avoir

Conserver la preuve de traitement de maniere durable pour les operations critiques. Les retries ne doivent jamais creer deux expéditions, deux factures ou deux remboursements.

13. Observabilite

Propager correlation_id, causation_id et trace_id. Ajouter traces OTel REST/gRPC/Kafka/RabbitMQ/provider adapters.

```
shipping_create_duration
carrier_api_errors_total
tracking_reconciliation_mismatch_total
provider_webhook_lag
invoice_generation_duration
notification_delivery_failures_total
rabbitmq_dlq_depth
return_processing_duration
refund_reconciliation_mismatch_total
```

Ajouter dashboards et alertes orientees symptomes/SLO, pas uniquement CPU/RAM.

14. Securite

- Secrets provider dans OpenBao.
- Aucun endpoint PostgreSQL/RabbitMQ interne expose sur Internet.
- mTLS workload via SPIFFE/SPIRE + Istio Ambient.
- Egress Internet via Istio Egress Gateway puis Squid, deny-by-default et allowlist.
- Validation backend obligatoire avant tout etat final presente au frontend.
- PII minimale dans logs/events ; ne jamais journaliser secrets, tokens ou donnees de paiement sensibles.

15. Tests obligatoires

Scenario	Resultat attendu
Order confirmed	Une expedition, une facture et les jobs de notification sont crees une seule fois.
Carrier timeout	Timeout borne, retry policy appliquee, aucun doublon d'expedition.
Webhook duplique	ACK idempotent, aucun doublon de timeline.
Postal/Jasmin down	Order unaffected ; message reste retryable/DLQ.
PDF worker down	Facture metier conservee ; generation reprend sans doublon.
Return approved	Etiquette retour et tracking demarrent.
Refund retry	Un seul remboursement financier.
Carrier webhook perdu	Reconciliation retrouve l'etat autoritative.
Invalid transition	Transition refusee et auditee.

16. CI/CD et livraison

Chaque service doit passer : gofmt, go vet, golangci-lint, tests unitaires, tests integration/testcontainers-go, race detector, build, Trivy, Syft/SBOM, Cosign, publication Harbor et deploiement FluxCD.

Ajouter les manifests/Helm/Kustomize necessaires, requests/limits, PDB, topologySpreadConstraints, anti-affinity, probes et NetworkPolicies.

Les services critiques utilisent Flagger si un canary apporte une valeur reelle.

17. Ordre d'implementation

1. Contrats/domain models + migrations
2. Shipping + carrier interface + fake carrier
3. Tracking + state machine + webhook security
4. Billing + invoice + async PDF job
5. Notification + RabbitMQ + Novu adapters
6. Returns + state machine
7. Refund / credit-note workflow
8. Reconciliation workers
9. Observability
10. Integration + E2E tests
11. Containers + CI/CD + GitOps
12. Documentation / runbooks

18. Definition of Done

```
order.confirmed.v1
-> shipment cree
-> label generee
-> tracking actif
-> facture emise
-> PDF genere de facon asynchrone
-> notifications programmees/envoyees
```

```
Puis:
customer return
-> demande validee backend
-> etiquette retour
-> suivi retour
-> reception
-> inspection
-> restock si eligible
-> remboursement unique
-> avoir unique
-> notification client
```

Le test E2E doit aussi demontrer les modes de panne : provider indisponible, webhook perdu/duplique, worker PDF down, Notification down et retry remboursement.

Le livrable n'est termine que lorsque les tests passent, les services compilent, les migrations sont reproductibles, les evenements sont idempotents et les scenarios de panne critiques ont ete verifies.

19. Regles de travail pour l'IA

Travaille par increments courts. Apres chaque increment : compile, execute les tests, corrige, puis factorise. Ne duplique pas la logique entre REST/gRPC/consumers. Ne cree pas d'abstraction sans usage concret. Conserve un diff coherent et documente les choix importants dans des ADR lorsque necessaire.

Si une hypothese manque mais ne bloque pas le developpement, choisis une valeur raisonnable et documente-la. Si elle touche une regle financiere, fiscale, legale ou une irreversible operation externe, ne l'invente pas : isole-la derriere une interface/configuration explicite.